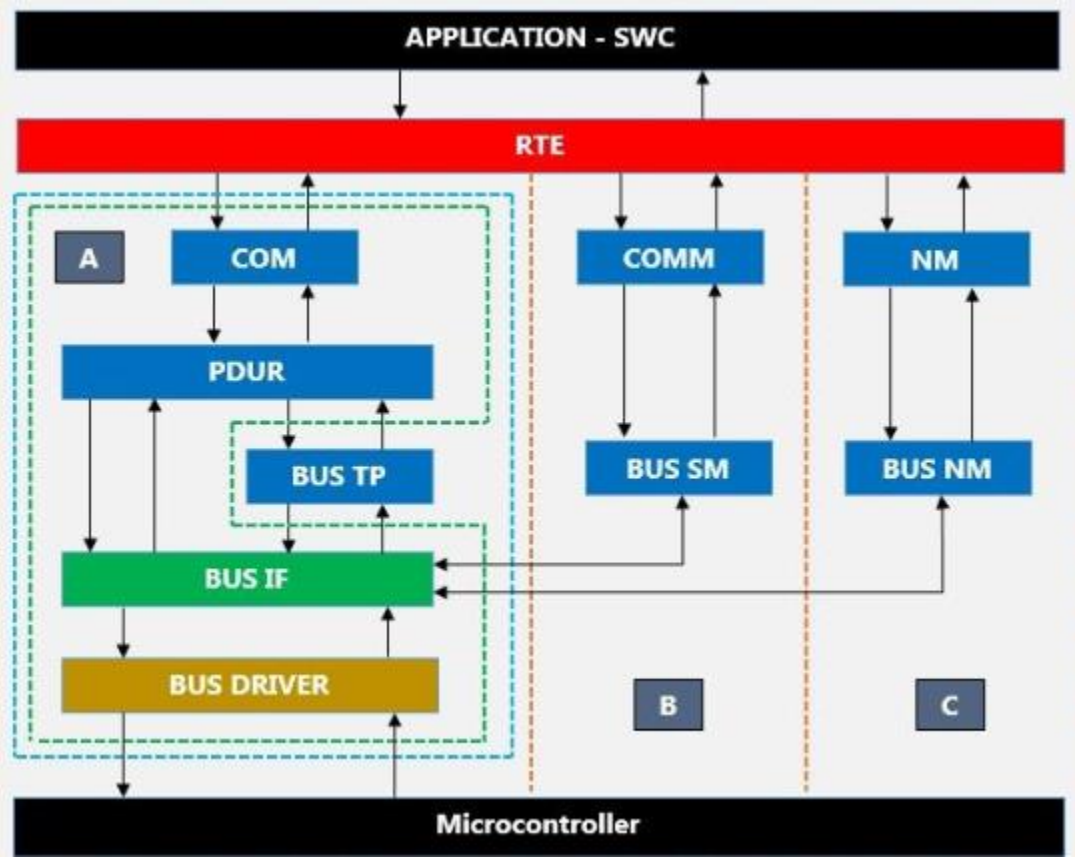
AUTOSAR COM Stack

Agenda

- Introduction
- CAN
- CAN Driver Module
- CAN IF Module
- CanTP Module
- PduR Module
- COM Module

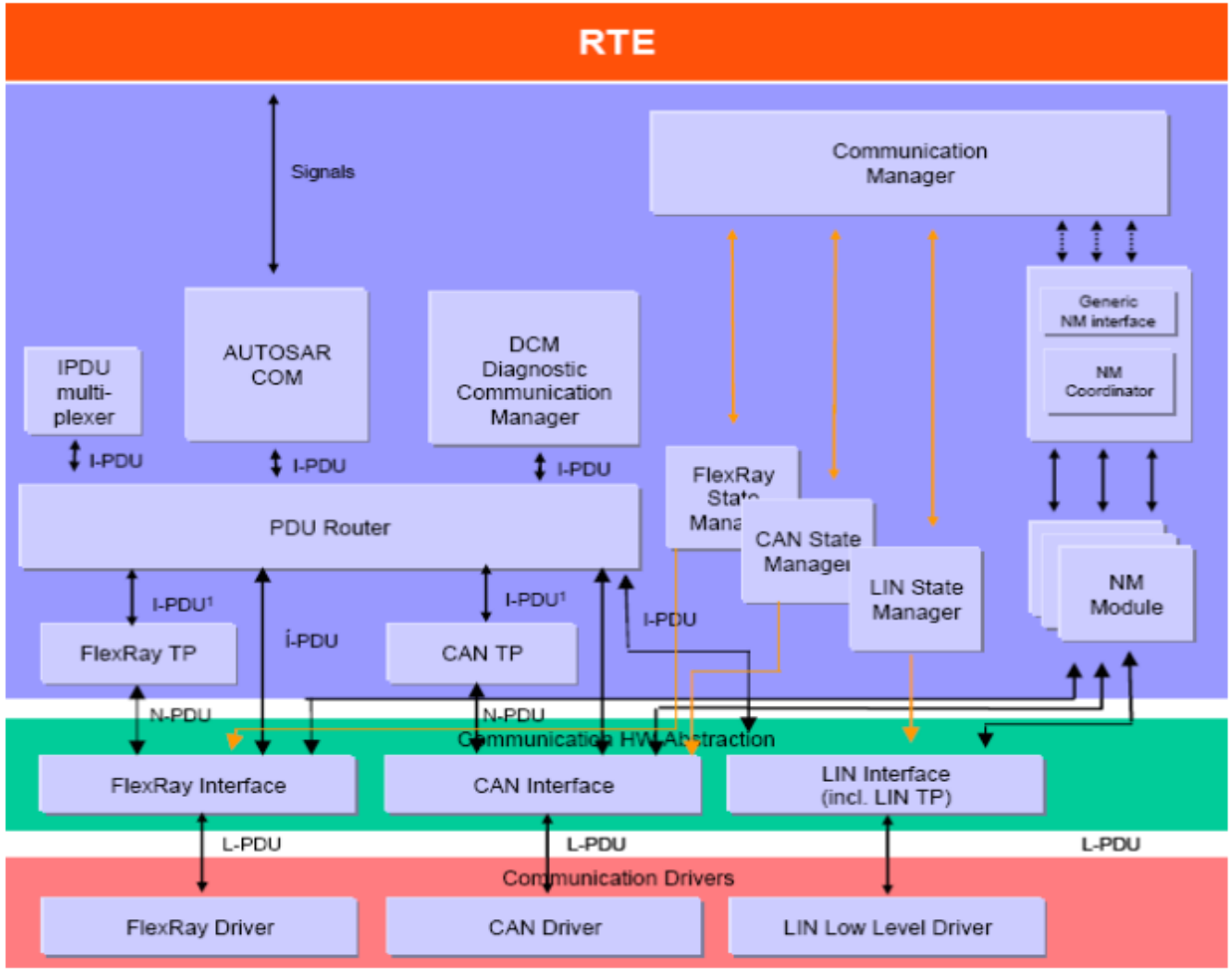
Com Stack



A. Data Transmission and reception Management

B. State Management

C. Network Mangagement


RTE
Signals
Communication Manager
AUTOSAR COM
DCM Diagnostic Communication Manager
IPDU multi-plexer
Generic NM interface
NM Coordinator
I-PDU
I-PDU
I-PDU
PDU Router
FlexRay Sta
Mana
CAN State Manager
LIN State Manager
NM Module
I-PDU¹
I-PDU¹
I-PDU
I-PDU
FlexRay TP
CAN TP
N-PDU
N-PDU
Communication HW Abstraction
FlexRay Interface
CAN Interface
LIN Interface (incl. LIN TP)
L-PDU
L-PDU
L-PDU
Communication Drivers
FlexRay Driver
CAN Driver
LIN Low Level Driver

PDU Flow through the Layered Architecture

SDU

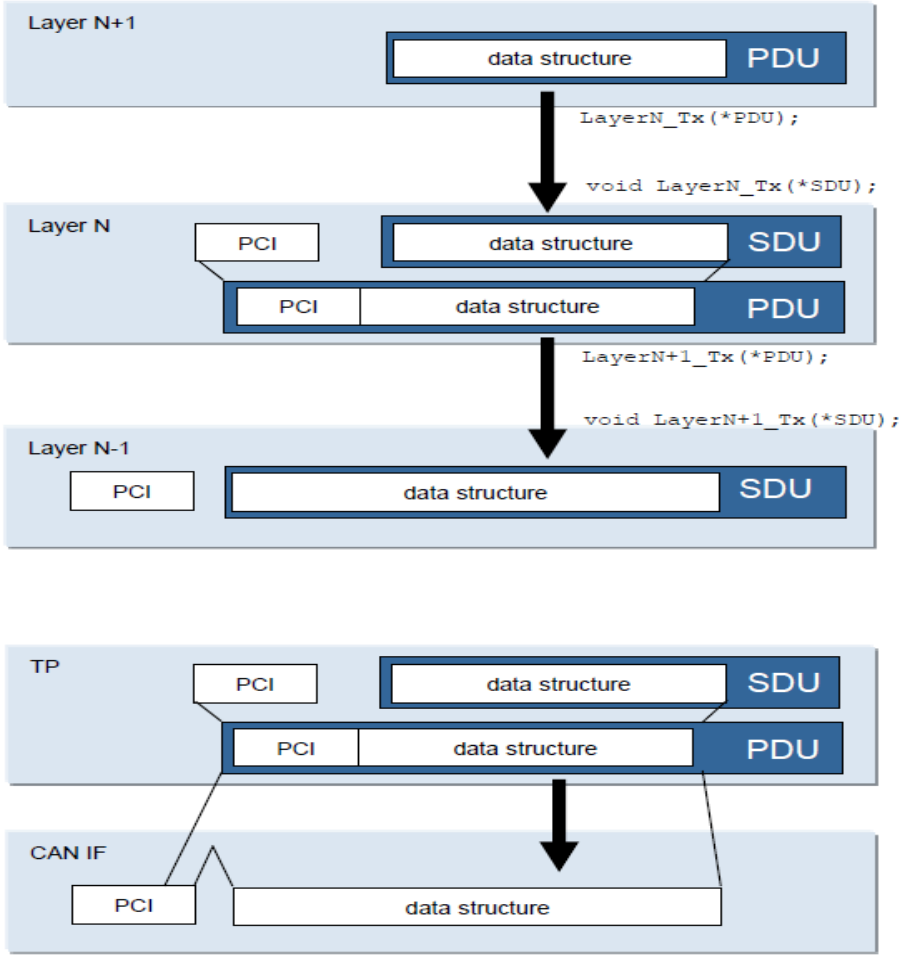
- SDU is the abbreviation of “Service Data Unit”. It is the data passed by an upper layer, with the request to transmit the data. It is as well the data which is extracted after reception by the lower layer and passed to the upper layer. A SDU is part of a PDU.

PCI

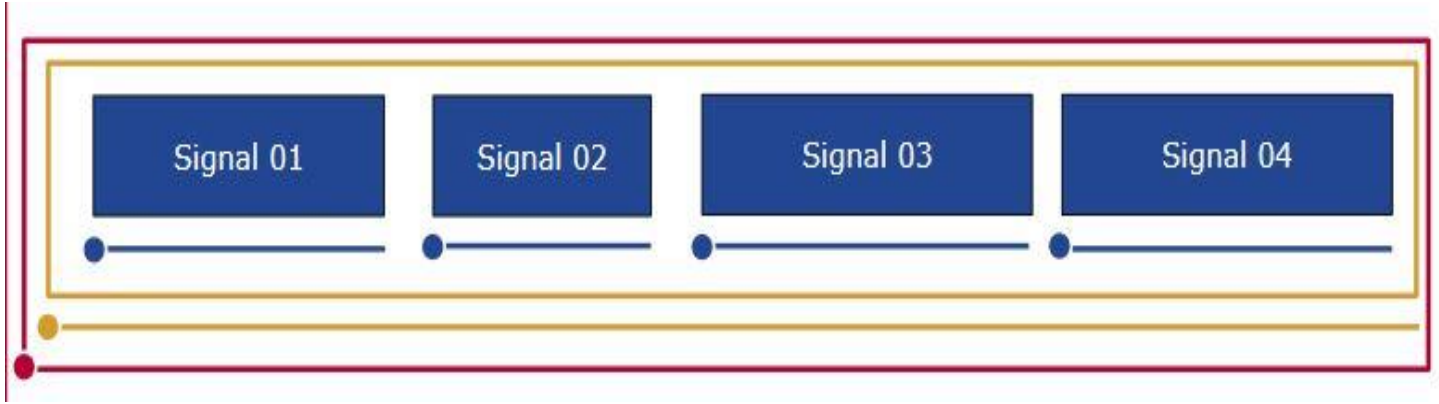
- PCI is the abbreviation of “Protocol Control Information”. This Information is needed to pass a SDU from one instance of a specific protocol layer to another instance. E.g. it contains source and target information. The PCI is added by a protocol layer on the transmission side and is removed again on the receiving side.

PDU

- PDU is the abbreviation of “Protocol Data Unit”. The PDU contains SDU and PCI. On the transmission side the PDU is passed from the upper layer to the lower layer, which interprets this PDU as its SDU.


Layer N+1
data structure
PDU
LayerN_Tx(*PDU);
void LayerN_Tx(*SDU);
Layer N
PCI
data structure
SDU
PCI
data structure
PDU
LayerN+1_Tx(*PDU);
void LayerN+1_Tx(*SDU);
Layer N-1
PCI
data structure
SDU
TP
PCI
data structure
SDU
PCI
data structure
PDU
CAN IF
PCI
data structure

Frame / PDU / Signal



- **SIGNAL**
- **PDU**
- **FRAME**

Frame / PDU / Signal

- A signal is the smallest atomic information that can be passed. Signals related to similar functionality/source/address can be grouped to make use of the complete 8 bytes available in the frame, such a group is called as Signal Group, similarly PDUs belonging to a similar functionality/cluster can be grouped and this group is called as PDU Group.

- PDU Consists of two parts, SDU and PCI. SDU is the actual data and PCI is the module specific information needed by the upper/lower layer, PCI can be data length/handle ID etc. PCI will be added by the sender and will be removed by the receiver on reception and it's PCI will be added and will be passed to the under lying receiver.

- A PDU's SDU can consist of a signal group (which may contain few signals grouped together), or a single signal. A frame can contain a single PDU or a PDU group.

- Since CAN allows only 8 byte data, the max length of a frame is only 8 bytes and so the max length a single PDU/Signal/PDU Group/Signal Group can only be 8 bytes.

SDU and PDU Naming Conventions

- The naming of PDUs and SDUs respects the following rules:
- For PDU: <***bus prefix> <layer prefix> - PDU***
- For SDU: <***bus prefix> <layer prefix> - SDU***
- The ***bus prefix and layer prefix are described in the following table:***

ISO Layer	Layer Prefix	AUTOSAR Modules	PDU Name	CAN / TTCAN prefix	LIN prefix	FlexRay prefix
Layer 6: Presentation (Interaction)	I	COM, DCM	I-PDU	N/A		
	I	PDU router, PDU multiplexer	I-PDU	N/A		
Layer 3: Network Layer	N	TP Layer	N-PDU	CAN SF CAN FF CAN CF CAN FC	LIN SF LIN FF LIN CF LIN FC	FR SF FR FF FR CF FR FC
Layer 2: Data Link Layer	L	Driver, Interface	L-PDU	CAN	LIN	FR

Components

PDU Router:

- Provides routing of PDUs between different abstract communication controllers and upper layers
- Scale of the Router is ECU specific (down to no size if e.g. only one communication controller exists)
- Provides TP routing on-the-fly. Transfer of TP data is started before full TP data is buffered

COM:

- Provides routing of individual signals or groups of signals between different I-PDUs

NM Coordinator:

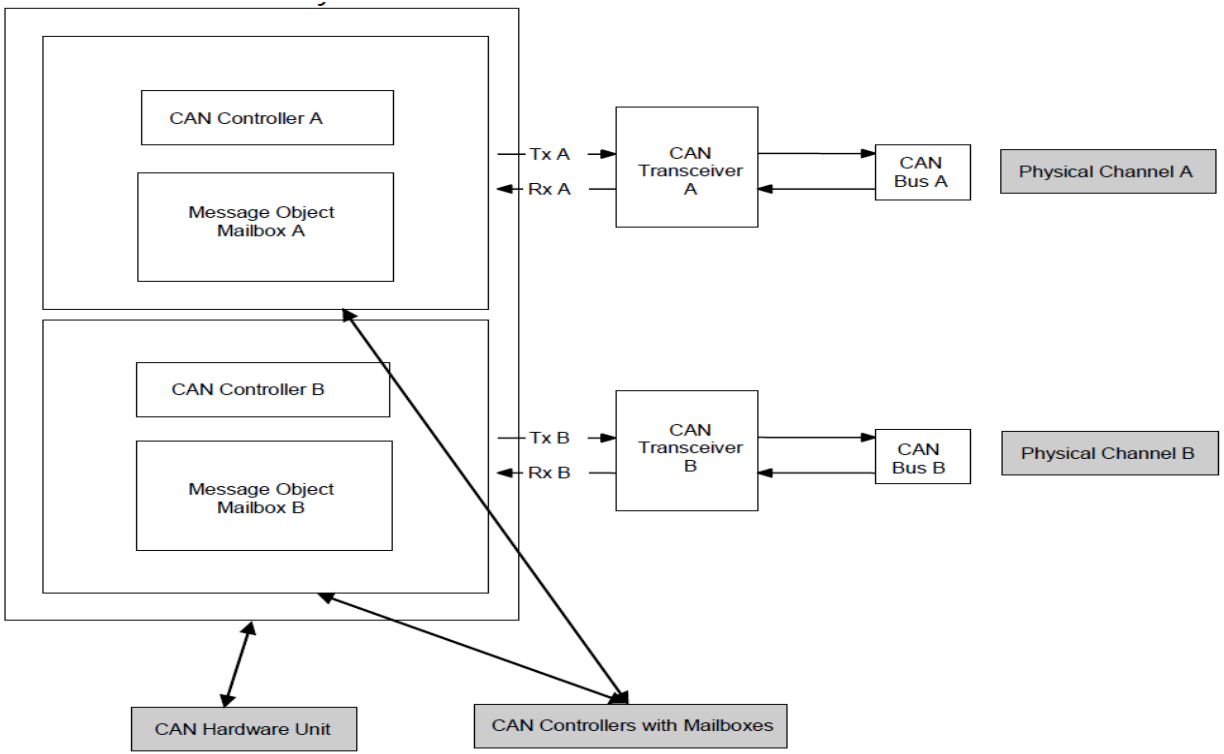
- Synchronization of Network States of different communication channels connected to an ECU via the network managements handled by the NM Coordinator
- Communication State Managers:
- Start and Shutdown the hardware units of the communication systems via the interfaces
- Control PDU groups

CAN Driver

- The Can module is part of the lowest layer, performs the hardware access and offers a hardware independent API to the upper layer.
- The only upper layer that has access to the Can module is the CanIf module
- The Can module provides services for initiating transmissions and calls the callback functions of the CanIf module for notifying events, independently from the hardware.
- Furthermore, it provides services to control the behavior and state of the CAN controllers that are belonging to the same CAN Hardware Unit.
- Several CAN controllers can be controlled by a single Can module as long as they belong to the same CAN Hardware Unit.
- A CAN driver is nothing more than a named entity that supports the basic I/O functions - read, write, get config, and set config. The device driver uses and manages interrupts from the device. While the interface is generic and device driver independent, the actual driver implementation is completely up to the device driver designer.

CAN Hardware Unit

- The CAN Hardware Unit combines one or several CAN controllers, which may be located on-chip or as external standalone devices of the same type, with common or separate Hardware Objects.



- **CAN controller** : A CAN controller serves exactly one physical channel.
- **CAN Hardware Unit** : A CAN Hardware Unit may consists of one or multiple CAN controllers of the same type and one or multiple CAN RAM areas. The CAN Hardware Unit is either on-chip, or an external device. The CAN Hardware Unit is represented by one CAN driver.
- **CAN L-PDU** : Data Link Layer Protocol Data Unit. Consists of Identifier, Data Length and Data (SDU).
- **CAN L-SDU** : Data Link Layer Service Data Unit. Data that is transported inside the L-PDU.
- **Hardware Object** : A CAN hardware object is defined as a PDU buffer inside the CAN RAM of the CAN hardware unit / CAN controller. A Hardware Object is defined as L-PDU buffer inside the CAN RAM of the CAN Hardware Unit.
- The **Hardware Receive Handle** (HRH) is defined and provided by the CAN Driver. Each HRH typically represents just one hardware object.
- The **Hardware Transmit Handle** (HTH) is defined and provided by the CAN Driver. Each HTH typically represents just one or multiple hardware objects that are configured as hardware transmit buffer pool.

- **Inner Priority Inversion** : Transmission of a high-priority L-PDU is prevented by the presence of a pending low-priority L-PDU in the same transmit hardware object.

- **L-PDU Handle** : The L-PDU handle is defined and placed inside the CanIf module layer. Typically each handle represents an L-PDU, which is a constant structure with information for Tx/Rx processing.

- **Outer Priority Inversion** : A time gap occurs between two consecutive transmit L-PDUs. In this case a lower priority L-PDU from another node can prevent sending the own higher priority L-PDU. Here the higher priority L-PDU cannot participate in arbitration during network access because the lower priority L-PDU already won the arbitration.

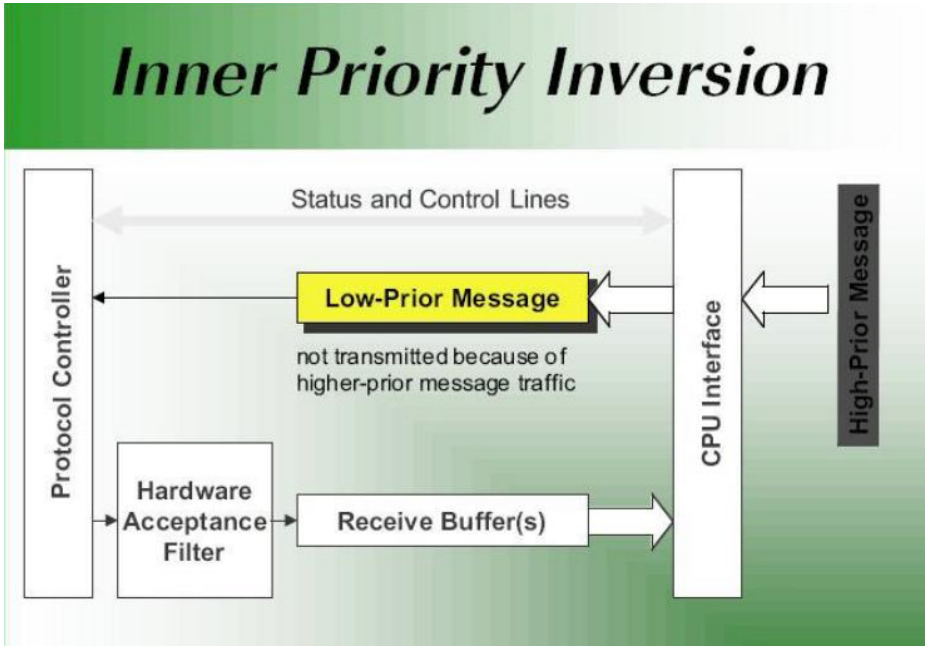
- **Physical Channel** : A physical channel represents an interface from a CAN controller to the CAN Network. Different physical channels of the CAN hardware unit may access different networks.

- **SFR** : Special Function Register. Hardware register that controls the controller behavior.

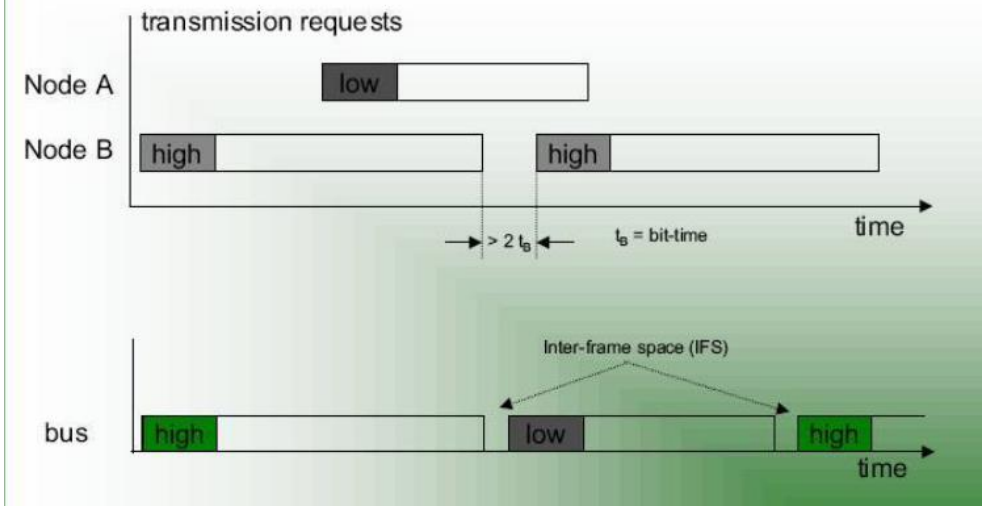
- **Priority** : The Priority of a CAN L-PDU is represented by the CAN Identifier. The lower the numerical value of the identifier, the higher the priority.

Inner Priority Inversion

- “If only a single transmit buffer is used inner priority inversion may occur. Because of low priority a message stored in the buffer waits until the ”traffic on the bus calms down”. During the waiting time this message could prevent a message of higher priority generated by the same microcontroller from being transmitted over the bus.”



Outer Priority Problem



Driver Services

- If the CAN controller is on-chip, the Can module shall not use any service of other drivers.
- The function Can_Init shall initialize all on-chip hardware resources that are used by the CAN controller. The only exception to this is the digital I/O pin configuration (of pins used by CAN), which is done by the port driver.
- The Mcu module shall configure register settings that are ‘shared’ with other modules.
- If an off-chip CAN controller is used, the Can module shall use services of other MCAL drivers (e.g. SPI).
- The Can module shall use the synchronous APIs of the underlying MCAL drivers and shall not provide callback functions that can be called by the MCAL drivers. Thus the type of connection between µC and CAN Hardware Unit has only impact on implementation and not on the API.

System Services

- In special hardware cases, the Can module shall poll for events of the hardware.
- The Can module shall use the OsCounter provided by the system service for timeout detection in case the hardware does not react in the expected time (hardware malfunction) to prevent endless loops.

Functional specification

- On L-PDU transmission, the Can module writes the L-PDU in an appropriate buffer inside the CAN controller hardware.
- On L-PDU reception, the Can module calls the RX indication callback function with ID, Data Length and pointer to L-SDU as parameter.
- The Can module provides an interface that serves as periodical processing function, and which must be called by the Basic Software Scheduler module periodically.
- Can module provides services to control the state of the CAN controllers. Bus-off and Wake-up events are notified by means of callback functions.
- The Can module shall implement the interrupt service routines for all CAN Hardware Unit interrupts that are needed.
- The Can module shall disable all unused interrupts in the CAN controller.
- The Can module shall reset the interrupt flag at the end of the ISR (if not done automatically by hardware).
- For CAN Hardware Units of different type, different Can modules shall be implemented.

L-PDU transmission

On L-PDU transmission, the Can module converts the L-PDU contents ID and Data Length to a hardware specific format (if necessary) and triggers the transmission.

Data mapping by CAN to memory is defined in a way that the CAN data byte which is sent out first is array element 0, the CAN data byte which is sent out last is array element 7 or 63 in case of CAN FD.

Several TX hardware objects with unique HTHs may be configured. The CanIf module provides the HTH as parameter of the TX request.

The function Can_Write shall store the swPduHandle that is given inside the parameter PduInfo until the Can module calls the CanIf_TxConfirmation for this request where the swPduHandle is given as parameter.

The Can module shall call CanIf_TxConfirmation to indicate a successful transmission.

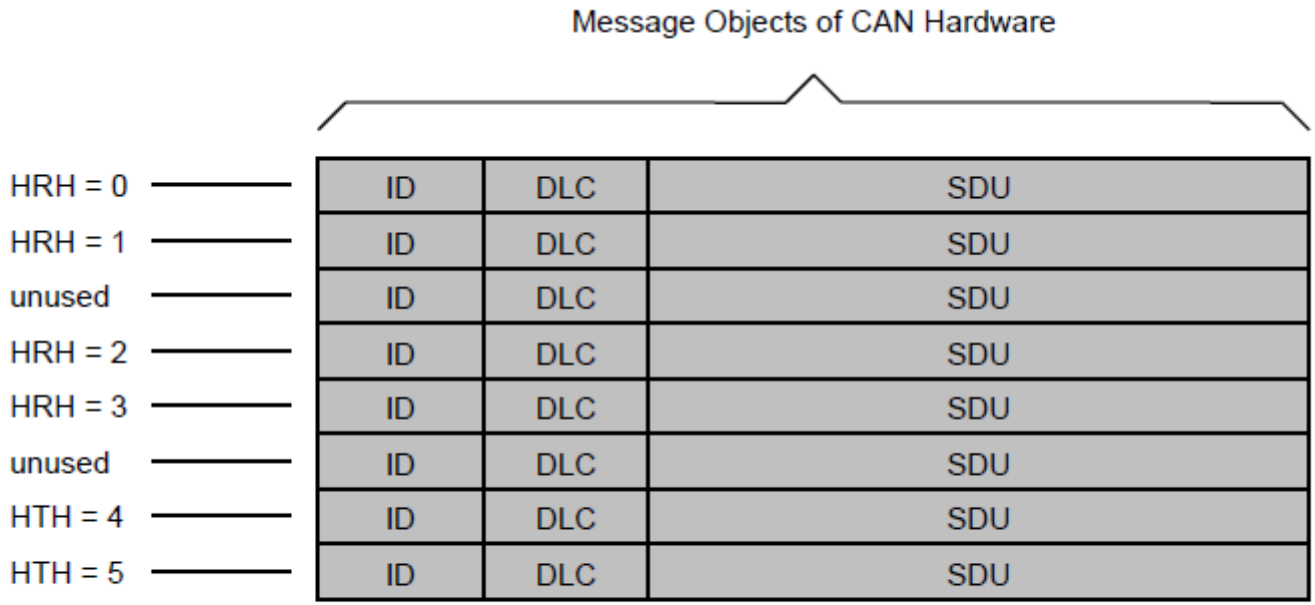


Figure 7-3: Example of assignment of HTHs and HRHs to the Hardware Objects. The numbering of HTHs and HRHs are implementation specific. The chosen numbering is only an example.

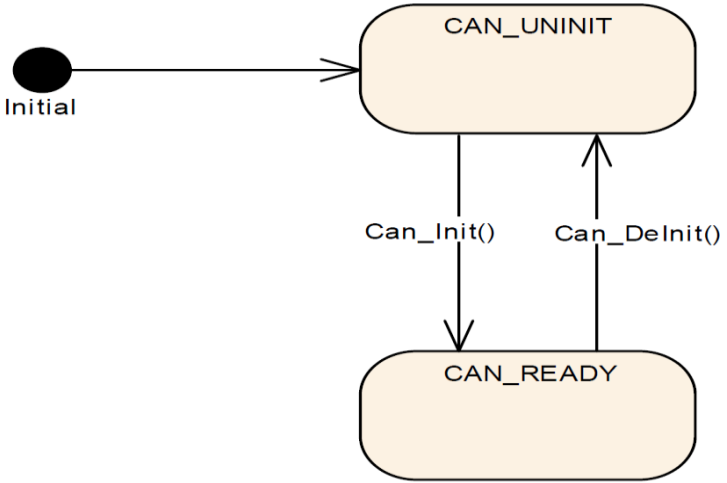
- L-PDU reception
- On L-PDU reception, the Can module shall call the RX indication callback function CanIf_RxIndication with ID, Hoh, abstract CanIf ControllerId in parameter Mailbox, and the Data Length and pointer to the L-SDU buffer in parameter PduInfoPtr.

- In case of an Extended CAN frame, the Can module shall convert the ID to a standardized format since the Upper layer (CANIF) does not know whether the received CAN frame is a Standard CAN frame or Extended CAN frame. In case of an Extended CAN frame, MSB of a received CAN frame ID needs to be made as ‘1’ to mark the received CAN frame as Extended.

- CanDrv shall indicate whether the received message is a conventional CAN frame or a CAN FD frame as described in Can_IdType.⌋

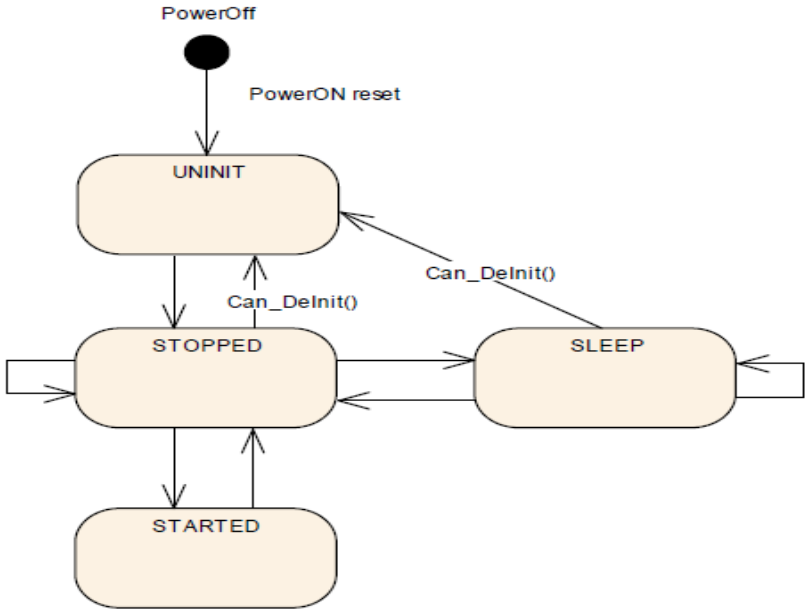
Driver State Machine

- After power-up/reset, the Can module shall be in the state CAN_UNINIT.
- The function Can_Init shall initialize all CAN controllers according to their configuration.
- Each CAN controller must then be started separately by calling the function Can_SetControllerMode (CAN_CS_STARTED).
- The function Can_DeInit shall change the module state to CAN_UNINIT before de-initializing all controllers inside the HW unit



CAN Controller State Machine

- Each CAN controller has complex state machines implemented in hardware. For simplification, the number of states is reduced to the following four basic states in this description: UNINIT, STOPPED, STARTED and SLEEP.



- The Can module offers the services
  - Can_Init
  - Can_SetBaudrate
  - Can_SetControllerMode.

- These services perform the necessary register settings that cause the required change of the hardware CAN controller state.

- There are two possibilities for triggering state changes by external events:
  - Bus-off event
  - HW wakeup event

- These events are indicated either by an interrupt or by a status bit that is polled in the Can_MainFunction_BusOff or Can_MainFunction_Wakeup.

- The Can module does the register settings that are necessary to fulfill the required behavior.
  Then it notifies the CanIf module with the corresponding callback function. The software state is then changed inside this callback function.

CAN controller state UNINIT

- The CAN controller is not initialized. All registers belonging to the CAN module are in reset state, CAN interrupts are disabled. The CAN Controller is not participating on the CAN bus.

CAN controller state STOPPED

- In this state the CAN Controller is initialized but does not participate on the bus. In addition, error frames and acknowledges must not be sent.

CAN controller state STARTED

- The controller is in a normal operation mode with complete functionality, which means it participates in the network.

CAN controller state SLEEP

- The hardware settings only differ from state STOPPED for CAN hardware that support a sleep mode (wake-up over CAN bus directly supported by CAN hardware).
- Can module shall set the controller to the SLEEP state from which the hardware can be woken over CAN Bus.
- When the CAN hardware does not support sleep mode and is triggered to transition into SLEEP state, the Can module shall emulate a logical SLEEP state from which it returns only, when it is triggered by software to transition into STOPPED state.]

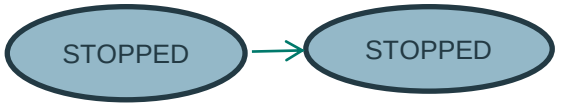
CAN Controller State Transitions

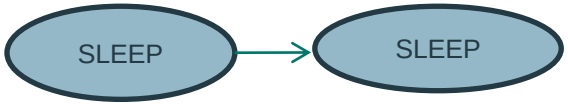
State transition caused by function Can_Init

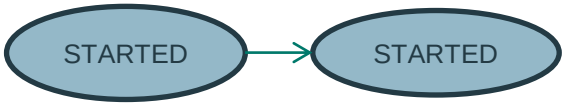


State transition caused by function Can_SetBaudrate

- software triggered by the function call Can_SetBaudrate changes the CAN controller configuration

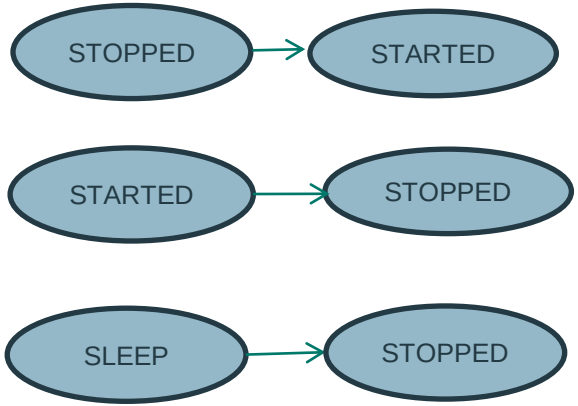






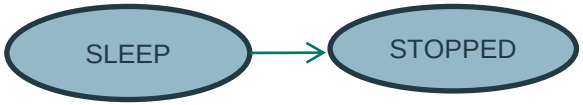
State transition caused by function Can_SetControllerMode

- The Can module notifies the upper layer (CanIf_ControllerModeIndication)
- The function Can_Mainfunction_Mode shall poll a flag of the CAN status register until the flag signals that the change takes effect and notify the upper layer with function CanIf_ControllerModeIndication about a successful state transition

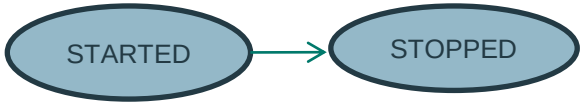


State transition caused by Hardware Events

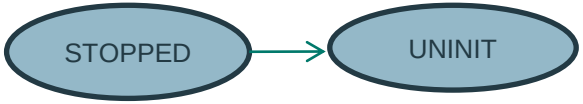
State transition caused by Hardware Wakeup

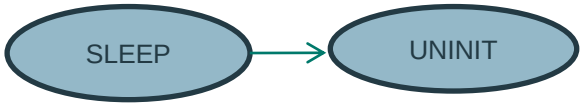


State transition caused by Bus-Off

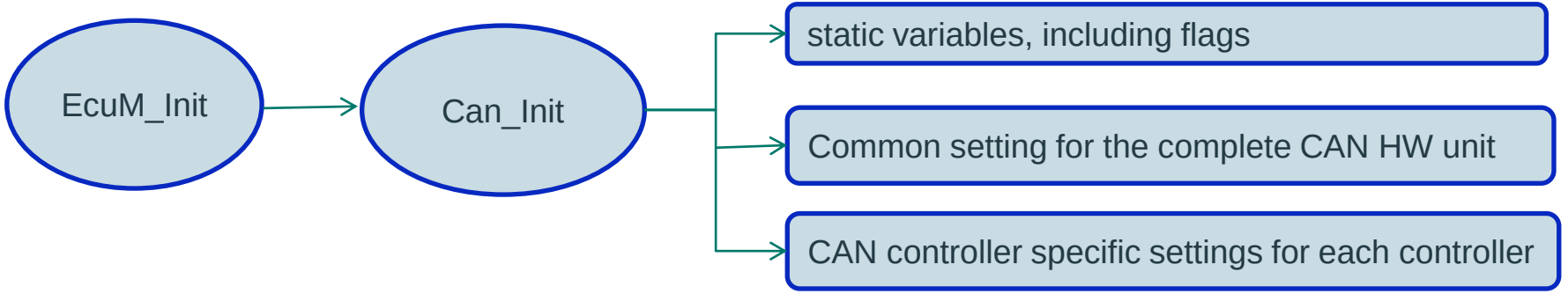


State transition caused by function Can_DeInit





Can Initialization

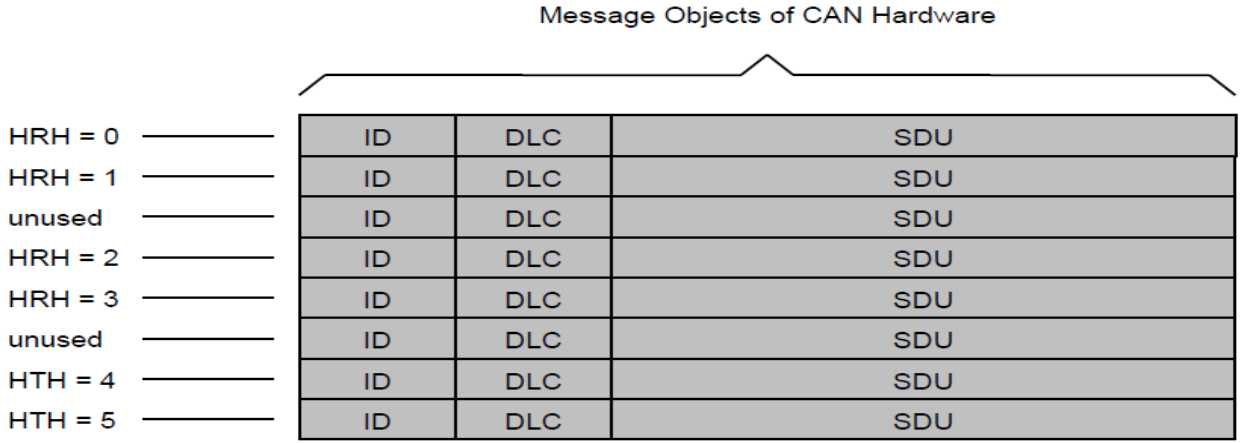


L-Pdu transmission

- On L-PDU transmission, the Can module converts the L-PDU contents ID and Data Length to a hardware specific format (if necessary) and triggers the transmission.

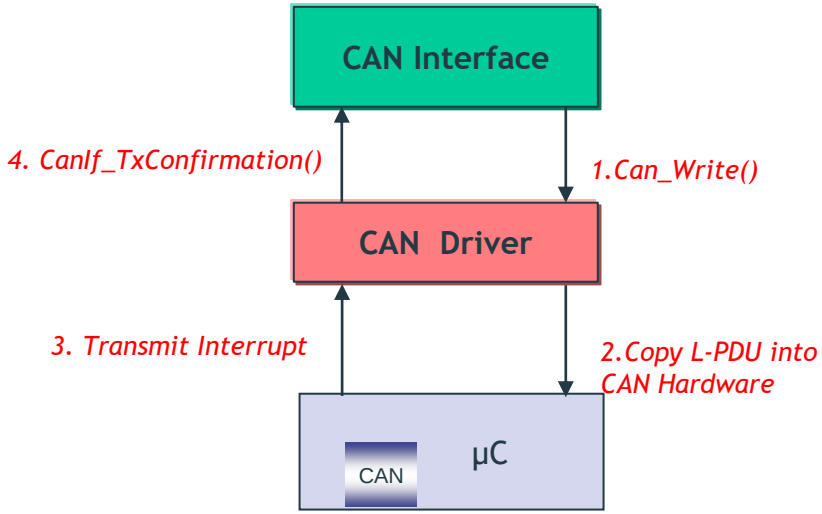
- Data mapping by CAN to memory is defined in a way that the CAN data byte which is sent out first is array element 0, the CAN data byte which is sent out last is array element 7 or 63 in case of CAN FD.

- Several TX hardware objects with unique HTHs may be configured. The CanIf module provides the HTH as parameter of the TX request.



- The function **Can_Write** shall store the swPduHandle (stored in PduInfo) until the Can module calls the **CanIf_TxConfirmation** for this request where the swPduHandle is given as parameter.

- The Can module shall call **CanIf_TxConfirmation** to indicate a successful transmission. It shall either be called by the **TX-interrupt** service routine of the corresponding HW resource or inside the Can_MainFunction_Write in case of **polling** mode.

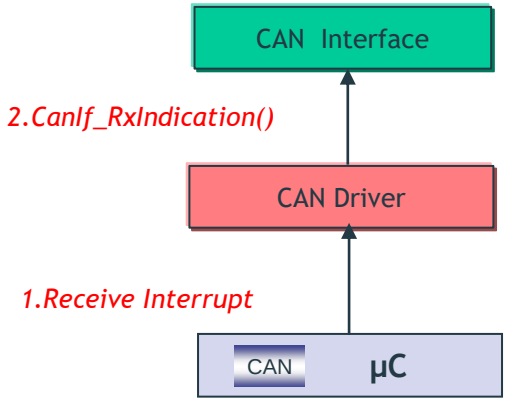


L-Pdu reception

- On L-PDU reception, the Can module shall call CanIf_RxIndication() with ID, Hoh, abstract CanIf ControllerId in parameter Mailbox, and the Data Length and pointer to the L-SDU buffer in parameter PduInfoPtr.

- The **RX-interrupt service routine** of the corresponding HW resource or the function Can_MainFunction_Read() in case of **polling** mode shall call the callback function CanIf_RxIndication.

- CanDrv shall indicate whether the received message is a conventional CAN frame or a CAN FD frame based on Can_IdType



Wakeup concept

The Can module handles wakeups that can be detected by the Can controller itself and not via the Can Trcv

- wakeup by interrupt
- wakeup by polling.

If the ISR for wakeup events is called, it shall call EcuM_CheckWakeup in turn.
The parameter passed to EcuM_CheckWakeup shall be the ID of the wakeup source referenced by the CanWakeupSourceRef configuration parameter

The ECU State Manager will then set up the MCU and call the Can module back via the Can Interface, resulting in a call to Can_CheckWakeup.

When wakeup events are detected by polling, the ECU State Manager will cyclically call Can_CheckWakeup via the Can Interface as before.

Can_CheckWakeup will check if there was a wakeup detected by a Can controller and return the result.

The CAN driver will then inform the ECU State Manager of the wakeup event via EcuM_SetWakeupEvent.

The wakeup validation to prevent false wakeup events, will be done by the ECU State Manager and the Can Interface afterwards

Notification concept

- The Can module offers only an event triggered notification interface to the CanIf module. Each notification is Represented by a callback function.
- The hardware events may be detected by an interrupt or by polling status flags of the hardware objects.
- The configuration of what is and is not polled by the Can module is internal to the driver, and not visible outside the module.
- The polling is done inside the CAN main functions (Can_MainFunction_xxx).
- The implementation of all callback functions shall be done as if the call context was the ISR.

Reentrancy issues

A routine must satisfy the following conditions to be reentrant:

- It uses all shared variables in an atomic way, unless each is allocated to a specific instance of the function.
- It does not call non-reentrant functions.
- It does not use the hardware in a non-atomic way.

The function CanIf_Transmit is re-entrant.

Therefore the function Can_Write needs to be implemented thread-safe (for example by using mutexes).

Can_EnableCanInterrupts and Can_DisableCanInterrupts may be called inside re-entrant functions. Therefore these functions also need to be reentrant. All other services don’t need to be implemented as reentrant functions.

The CAN main functions (i.e. Can_MainFunction_Read) shall not be interrupted by themselves. Therefore these CAN main functions are not reentrant.

Hardware Timestamping

Hardware-based timestamping, if supported by the CAN controller, can be used e.g. to enhance The precision of a synchronized time-base on CAN.

- *Can_GetCurrentTime*
- *Can_EnableEgressTimeStamp*
- *Can_GetEgressTimeStamp*
- *Can_GetIngressTimeStamp*

These APIs need to be enabled by the configuration parameter **CanGlobalTimeSupport**.

Error classification

Development Errors

Type of error	*Related error code*	*Error value*
API Service called with wrong parameter	CAN_E_PARAM_POINTER	0x01
API Service called with wrong parameter	CAN_E_PARAM_HANDLE	0x02
API Service called with wrong parameter	CAN_E_PARAM_DATA_LENGTH	0x03
API Service called with wrong parameter	CAN_E_PARAM_CONTROLLER	0x04
API Service used without initialization	CAN_E_UNINIT	0x05
Invalid transition for the current mode	CAN_E_TRANSITION	0x06
Parameter Baudrate has an invalid value	CAN_E_PARAM_BAUDRATE	0x07
API service called with invalid PDU ID	CAN_E_PARAM_LPDU	0x0A

Runtime Errors

Type of error	*Related error code*	*Error value*
Received CAN message is lost	CAN_E_DATALOST	0x01

Can Fd Support

For performance reasons some CAN controllers allow to use a Flexible Data-Rate feature called CAN FD

Indicated during the arbitration phase it is possible to switch to a higher baud rate during payload and CRC. This second baud rate has to be configured by extending CanControllerBaudrateConfig with CanControllerFdBaudrateConfig.

If a baud rate is active which has a CAN FD configuration (see CanControllerFdBaudrateConfig) the CAN FD feature Is enabled for this controller.

This can be achieved on frame level by using the two most significant bits of the CanId (using Can_IdType) passed during Can_Write to indicate which kind of frame shall be used.

CAN FD also supports an extended payload which allows the transmission of up to 64 bytes.

Can Api's

Can_Init	void Can_Init (const Can_ConfigType* Config)
Can_GetVersionInfo	void Can_GetVersionInfo (Std_VersionInfoType*versioninfo)
Can_DeInit	void Can_DeInit (void)
Can_SetBaudrate	Std_ReturnType Can_SetBaudrate (uint8 Controller, uint16 BaudRateConfigID)
Can_SetControllerMode	Std_ReturnType Can_SetControllerMode (uint8 Controller, Can_ControllerStateType Transition)
Can_DisableControllerInterrupts	void Can_DisableControllerInterrupts (uint8 Controller)
Can_EnableControllerInterrupts	void Can_EnableControllerInterrupts (uint8 Controller)
Can_CheckWakeup	Std_ReturnType Can_CheckWakeup (uint8 Controller)
Can_GetControllerErrorState	Std_ReturnType Can_GetControllerErrorState (uint8 ControllerId, Can_ErrorStateType* ErrorStatePtr)
Can_GetControllerMode	Std_ReturnType Can_GetControllerMode (uint8 Controller, Can_ControllerStateType* ControllerModePtr)
Can_GetControllerRxErrorCounter Can_GetControllerTxErrorCounter	Std_ReturnType Can_GetControllerRxErrorCounter (uint8 ControllerId, uint8* RxErrorCounterPtr) Std_ReturnType Can_GetControllerTxErrorCounter (uint8 ControllerId, uint8* TxErrorCounterPtr)
Can_Write	Std_ReturnType Can_Write (Can_HwHandleType Hth, const Can_PduType* PduInfo)

Call-back notifications

<LPDU_CalloutName>	boolean <LPDU_CalloutName> (uint8 Hrh, Can_IdType CanId, uint8 CanDataLegth, const uint8* CanSduPtr)

where <LPDU_CalloutName> has to be substituted with the concrete L-PDU callout name which is configurable

Enabling/Disabling wakeup notification

- Icu_EnableNotification
- Icu_DisableNotification

Scheduled functions

Can_MainFunction_Write (void)	polling of TX confirmation when CAN_TX_PROCESSING is set to POLLING.
Can_MainFunction_Read (void)	RX indications when CAN_RX_PROCESSING is set to POLLING.
Can_MainFunction_BusOff (void)	polling of bus-off events that are configured statically as 'to be polled'.
Can_MainFunction_Wakeup (void)	polling of wake-up events that are configured statically as 'to be polled'.
Can_MainFunction_Mode (void)	polling of CAN controller mode transitions

CAN Configuration

Included Containers		
Container Name	**Multiplicity**	**Scope / Dependency**
CanConfigSet	1	This container contains the configuration parameters and sub containers of the AUTOSAR Can module.
CanGeneral	1	This container contains the parameters related each CAN Driver Unit.

CanGeneral Configurations

CanDevErrorDetect	True / False
CanEnableSecurityEventReporting	True / False
CanGlobalTimeSupport	True / False
CanIndex	0
CanLPduReceiveCalloutFunction	name of a callout function that is called after a successful reception of a received CAN Rx L-PDU
CanMainFunctionBusoffPeriod	0.01 (10 ms)
CanMainFunctionModePeriod	0.01 (10 ms)
CanMainFunctionWakeupPeriod	0.01 (10 ms)
CanMultiplexedTransmission	ON/OFF
CanSetBaudrateApi	True / False (If true , api is supported)
CanTimeoutDuration	1
CanVersionInfoApi	Switches the Can_GetVersionInfo() API ON or OFF.
CanEcucPartitionRef	0
CanOsCounterRef	Os_Counter_1

Can Controller Config

CanBusoffProcessing	INTERRUPT / POLLING (API Can_MainFunction_BusOff())
CanControllerActivation	Defines if a CAN controller is used in the configuration.
CanControllerBaseAddress	Specifies the CAN controller base address. (0 .. 4294967295)
CanControllerId	0 (0..255)
CanRxProcessing	INTERRUPT / POLLING/ MIXED (API Can_MainFunction_Read())
CanTxProcessing	INTERRUPT / POLLING/ MIXED (API Can_MainFunction_Write())
CanWakeupProcessing	INTERRUPT / POLLING (API Can_MainFunction_Wakeup())
CanWakeupSupport	True/ False
CanControllerDefaultBaudrate	Reference to CanControllerBaudrateConfig
CanControllerEcucPartitionRe	0
CanCpuClockRef	Reference to McuClockReferencePoint
CanWakeupSourceRef	Symbolic name reference to EcuMWakeupSource

CanControllerBaudrateConfig

CanControllerBaudRate	Specifies the baudrate of the controller in kbps.
CanControllerBaudRateConfigID	identifies a specific baud rate configuration
CanControllerPropSeg	Specifies propagation delay in time quantas.
CanControllerSeg1	Specifies phase segment 1 in time quantas.
CanControllerSeg2	Specifies phase segment 2 in time quantas.
CanControllerSyncJumpWidth	Specifies the synchronization jump width for the controller in time quantas.

CanControllerFdBaudRate

Same configurations as above for FD

CanHardwareObject

CanFdPaddingValue	0
CanHandleType	BASIC CAN : For several L-PDUs are hadled by the hardware object FULL CAN: For only one L-PDU (identifier) is handled by the hardware object
CanHardwareObjectUsesPolling	Enables polling of this hardware object.
CanHwObjectCount	Number of hardware objects used to implement one HOH. In case of a HRH this parameter defines the number of elements in the hardware FIFO or the number of shadow buffers, in case of a HTH it defines the number of hardware objects used for multiplexed transmission or for a hardware FIFO used by a FullCAN HTH.
CanIdType	EXTENDED / MIXED / STANDARD
CanObjectId	Holds the handle ID of HRH or HTH.
CanObjectPayloadLength	Specifies the maximum L-PDU payload length in bytes the hardware object can store. (8, 64,48,32,24,20,16,12)
CanObjectType	RECEIVE / TRANSMIT
CanControllerRef	Reference to CAN Controller to which the HOH is associated to.
CanMainFunctionRWPeriodRef	Reference to CanMainFunctionPeriod.